CONFIDENTIAL — SECURITY ASSESSMENT

Advanced Vulnerability Assessment & Penetration Test Report

OWASP Juice Shop — Local Test Instance

Prepared by	Tanmay Sarkar — B.Tech CSE, Tezpur University
Target	OWASP Juice Shop (http://127.0.0.1:3000)
Assessment type	Manual VAPT — Burp Suite Community Edition (Proxy + Repeater)
Testing window	26–27 October 2026 (approx. 00:06–20:13 local)
Report date	28 October 2026
Report version	v2.0 — updated with supplementary address-endpoint testing

This report documents authorized testing performed against a locally hosted, deliberately vulnerable training application. Findings and recommendations are provided for educational and portfolio purposes.

1. Executive Summary

This report documents an advanced Vulnerability Assessment and Penetration Test (VAPT) performed against **OWASP Juice Shop**, following a business-logic and attack-chain testing methodology (per OWASP WSTG guidance) rather than relying solely on automated scanning. Testing used Burp Suite Community Edition (Proxy + Repeater) to intercept, replay, and tamper with HTTP requests across the product search, shopping basket, order, address, and administrative API surfaces, across two testing sessions on 26–27 October 2026.

Ten distinct tests were carried out and are documented in Section 5 with request/response evidence. The most significant findings are that the **admin application-configuration endpoint** returned a full, sensitive configuration object to a request carrying only a normal authenticated user's Bearer token (F-1), and that the **address endpoint exposed other users' personal data through a confirmed Insecure Direct Object Reference (IDOR)** — a single authenticated session was able to retrieve full name, phone number, and postal address records belonging to a different account simply by changing a numeric ID in the URL (F-8). The application was also observed to leak internal server implementation details — Node.js/Express/Sequelize file paths and ORM internals — in two independent, unhandled-exception scenarios (F-2, F-10).

A follow-up session specifically targeted the */api/Addresss* endpoint using two test accounts and sequential ID enumeration. This confirmed the IDOR described above and also surfaced an inconsistency in route-level protection: a correctly spelled, singular */api/Address/{id}* path rejected the same request with a generic anti-tampering message, while the misspelled */api/Addresss/{id}* path actually used by the application performed no such check (F-9).

Positive controls were also confirmed: requests to */rest/basket/{id}* without an Authorization header were correctly rejected with 401 Unauthorized, and a basic SQL-injection probe (') against the search parameter did not produce a database error or data leak.

2. Target and Testing Environment

Target application	OWASP Juice Shop (juice-sh.op) — a deliberately vulnerable training platform
Host / URL	http://127.0.0.1:3000 (local instance)
Application identity	Reported by the app's own /rest/admin/application-configuration response — name "OWASP Juice Shop", domain juice-sh.op
Proxy / testing tool	Burp Suite Community Edition v2026.7.2 (Proxy + Repeater)
Browser	Chromium-based browser (Chrome/150.0.0.0) routed through Burp proxy
Tester	Tanmay Sarkar, B.Tech CSE, Tezpur University (Roll No. CSB23063)
Test accounts used	Two registered test accounts — tanmaytest2026@example.com (UserId 25) and tanmaytest2@example.com (UserId 26)
Testing window	26–27 October 2026 (approx. 00:06–20:13 local capture timestamps, across an initial session and a supplementary follow-up session)

All testing was performed against a locally hosted, deliberately vulnerable instance intended for security training, consistent with the scope defined for this assessment.

3. Scope

The following functionality was in scope and exercised with Burp Suite Repeater:

- Product search (*/rest/products/search*) — query-parameter handling and input fuzzing
- Product detail retrieval (*/api/Products/{id}*) — parameter tolerance
- Shopping basket (*/rest/basket/{id}*) — object-level authorization (IDOR) and authentication enforcement, including malformed-path handling
- Basket items (*/api/BasketItems*) — business-logic constraint testing (duplicate item submission)
- Administrative configuration (*/rest/admin/application-configuration*) — access-control testing
- Address endpoint (*/api/Addresss* and */api/Address/{id}*) — object creation, listing, and object-level authorization (IDOR) testing across two accounts
- Session cookies, continuation codes, and Authorization: Bearer tokens observed across requests

Out of scope: payment processing, file upload, and third-party integrations were not exercised in this round of testing.

4. Methodology

1. **Proxy configuration —** browser traffic was routed through Burp Suite so that every request/response between the browser and the Juice Shop backend could be observed.
2. **Application mapping —** core features (product search, basket, checkout, account, address book) were exercised normally to populate Burp's HTTP history and identify interesting endpoints, IDs, and tokens.
3. **Repeater-based tampering —** selected requests were sent to Repeater, where parameters (search terms, object IDs, headers) were modified and replayed without repeating the action in the browser.
4. **Response comparison —** for each modified request, the status code, response body, and error content were compared against the original/expected behavior.
5. **Authorization validation —** requests were replayed with and without the Authorization header, and with tokens belonging to different test accounts against sequential object IDs, to check whether the server enforced access control independently of client input.
6. **Evidence capture —** the original request, modified request, and full response were screenshotted for every test, per the evidence standard below.

Consistent with standard testing practice, a finding was only recorded as confirmed when the server's response demonstrated the unauthorized action or disclosure; where the server correctly rejected a request (e.g. with a 401), this was logged as a working control rather than a vulnerability.

5. Findings

F-1: Admin configuration endpoint accessible with a standard user token (Broken Access Control)

Request: GET /rest/admin/application-configuration (also reachable as /api/admin/application-configuration), sent with only a standard authenticated user's Authorization: Bearer token (test account tanmaytest2@example.com) and no elevated role.

Response: 200 OK, 23,577 bytes — the full application configuration object, including the application name ("OWASP Juice Shop"), domain (juice-sh.op), server base URL, theme, feature flags (showVersionNumber, showGitHubLinks, localBackupEnabled), the privacy contact email, custom metrics prefix, and chatbot configuration.

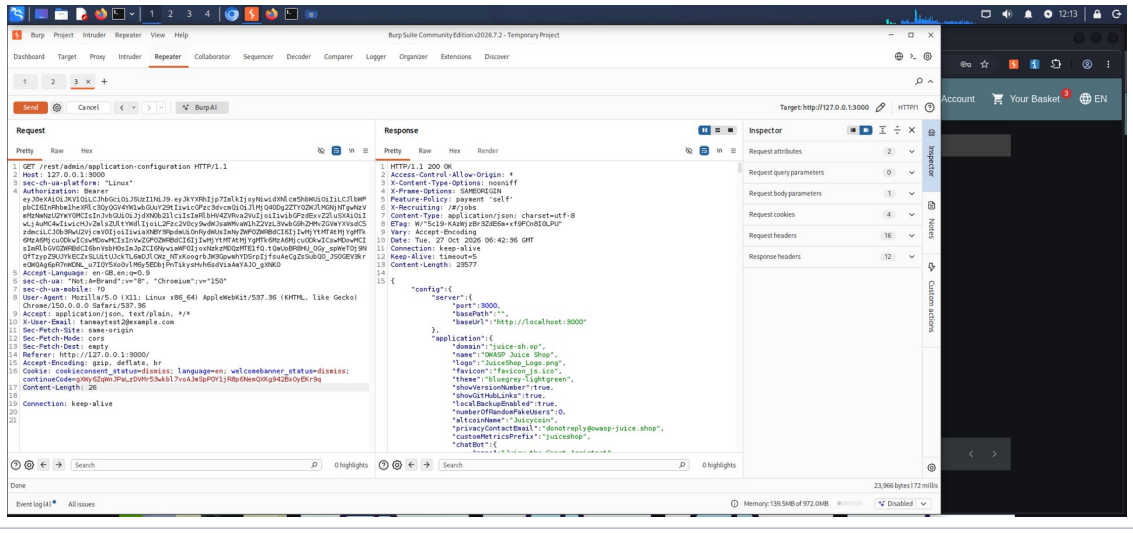


Figure 1 — Admin configuration endpoint returning full config to a standard user token

Assessment: The endpoint path (/admin/…) implies restricted/administrative access, but the response was returned to a session with no indication of an elevated role. No authorization error, role check, or redirect was observed. This is a **Broken Access Control / sensitive information disclosure** finding — the server trusted the presence of any valid token rather than verifying the caller's privilege level for an admin-scoped resource.

F-2: Verbose server error discloses stack trace on business-logic violation

Request: POST /api/BasketItems with body {"ProductId":1,"BasketId":6}, attempting to add a product that already existed in that basket.

Response: 500 Internal Server Error containing a full SequelizeUniqueConstraintError, including the internal file paths (/juice-shop/node_modules/sequelize/…), SQL dialect internals, the failing model instance, and the raw ProductId/BasketId values.

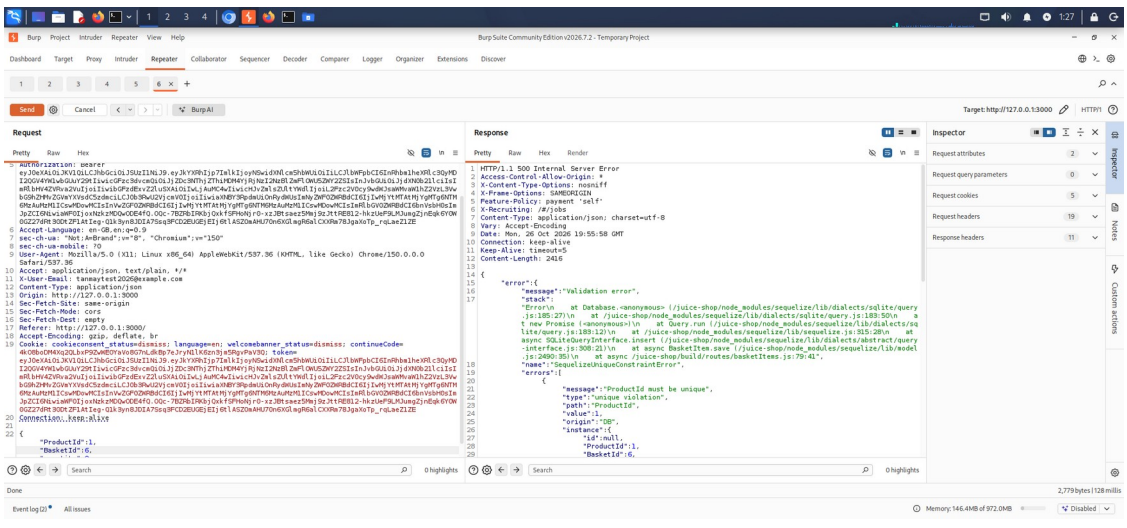


Figure 2 — 500 error exposing a full Sequelize stack trace for a duplicate-item business rule

Assessment: Positively, the application does enforce a business rule preventing a duplicate ProductId in the same basket (correct behavior). However, instead of returning a clean, generic 4xx validation error, the server leaks internal implementation details — ORM name, file-system paths, and query internals — in the HTTP response. This is an **Information Disclosure via verbose error handling** finding, which aids an attacker in fingerprinting the backend stack and refining further attacks.

F-3: Empty search query returns the entire product catalog

Request: GET /rest/products/search?q= (empty query string).

Response: 200 OK, 16,563 bytes — the complete, unfiltered product catalog, including items not shown in normal browsing (e.g. a product named "Apple Pomace" whose description embeds raw HTML: Can be sent back to us for recycling.).

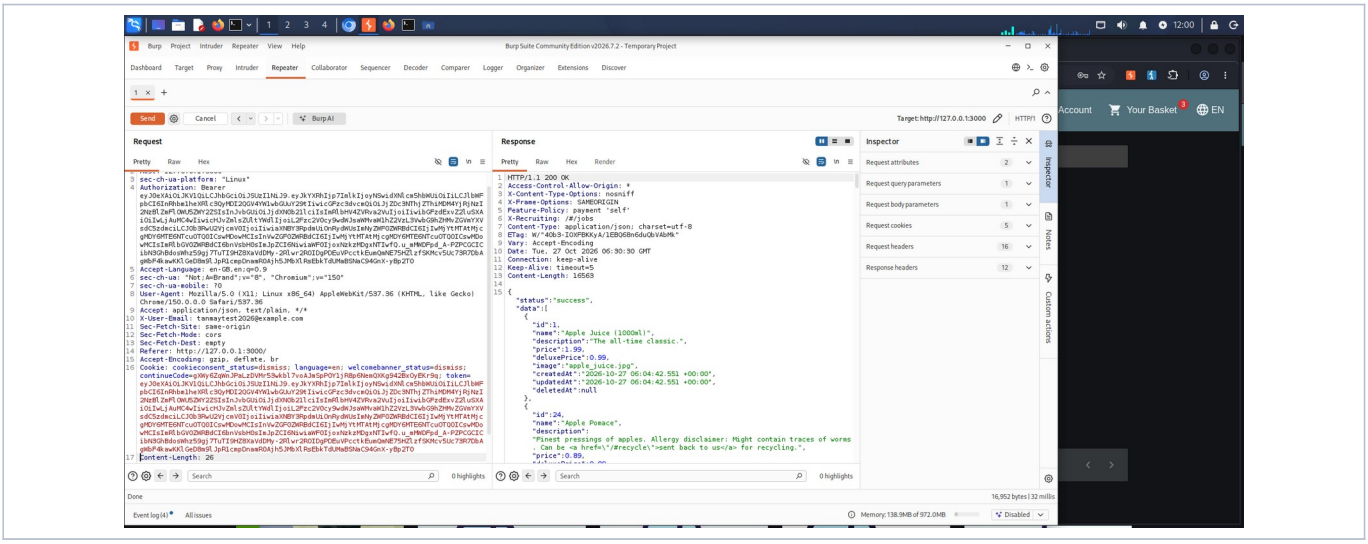


Figure 3 — Empty search query returning the full product catalog including HTML-bearing descriptions

Assessment: An empty or missing search term should ideally be rejected or paginated; instead it short-circuits to "return everything," which is an unbounded-response / information-disclosure pattern and an inefficient one at scale. Separately, product descriptions containing unescaped HTML anchor tags are returned as-is in the JSON response — if this content is rendered without output encoding on the client, it represents a **Stored XSS** risk (flagged for follow-up; client-side rendering was not verified in this round).

F-4: Authentication correctly enforced on the basket endpoint (positive control)

Request: GET /rest/basket/6 sent with the Authorization header stripped.

Response: 401 Unauthorized — "UnauthorizedError: No Authorization header was found".

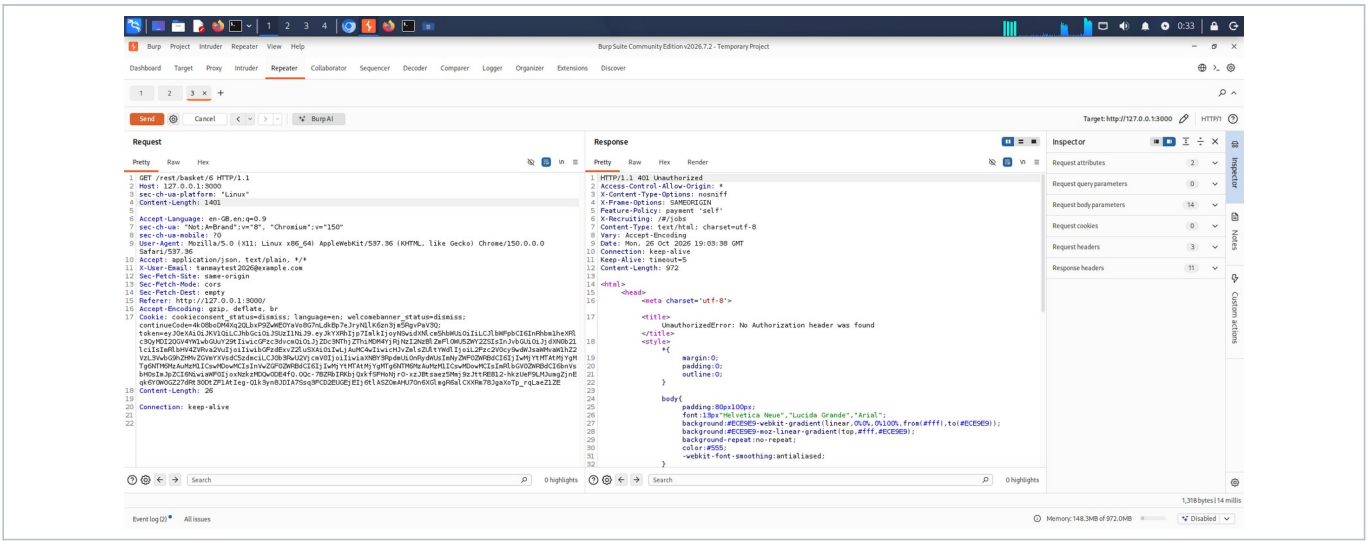


Figure 4 — 401 Unauthorized returned when the Authorization header is removed

Assessment: This control is working as intended: the server rejects basket access when no bearer token is supplied. No finding.

F-5: SQL-injection probe on search parameter — no error disclosure (positive control)

Request: GET /rest/products/search?q=%27 (a single quote, ', the classic SQLi probe character).

Response: 200 OK with an empty data: [] array — no SQL error, stack trace, or anomalous behavior.

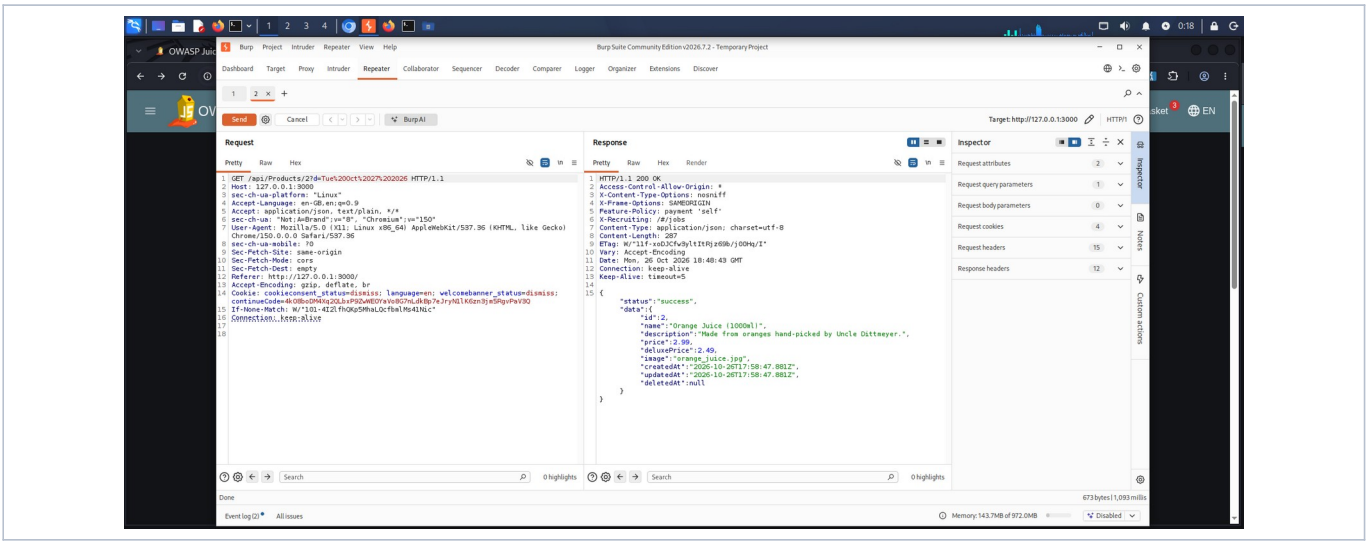


Figure 5 — Single-quote SQL injection probe handled safely with an empty result, no DB error

Assessment: The search parameter appears to be parameterized/sanitized against this basic probe; no injection was demonstrated. This should ***not*** be read as a full SQLi clearance — only one payload class was tested — but no finding is recorded from this specific test.

F-6: Basket object-level access — IDOR test (inconclusive at the time, now a priority retest in light of F-8)

Requests: GET /rest/basket/6 and GET /rest/basket/7 were each tested with Bearer tokens tied to two different test accounts. Basket 6 consistently returned data for UserId 25; basket 7 consistently returned data for UserId 26.

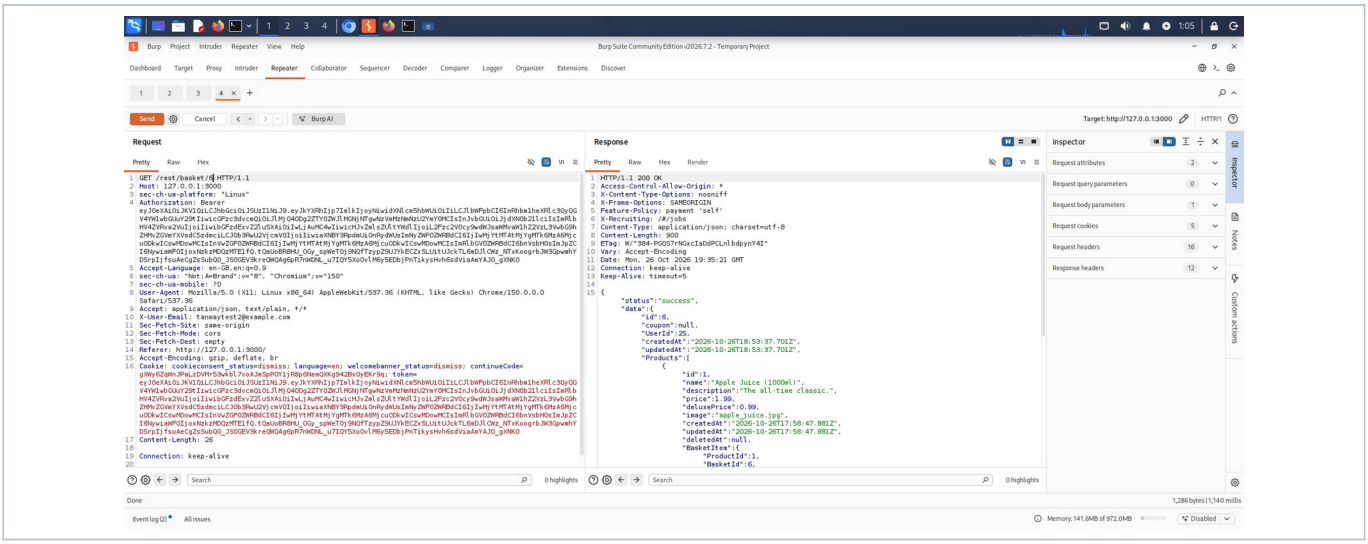


Figure 6 — Basket 6 returning UserId 25's cart contents

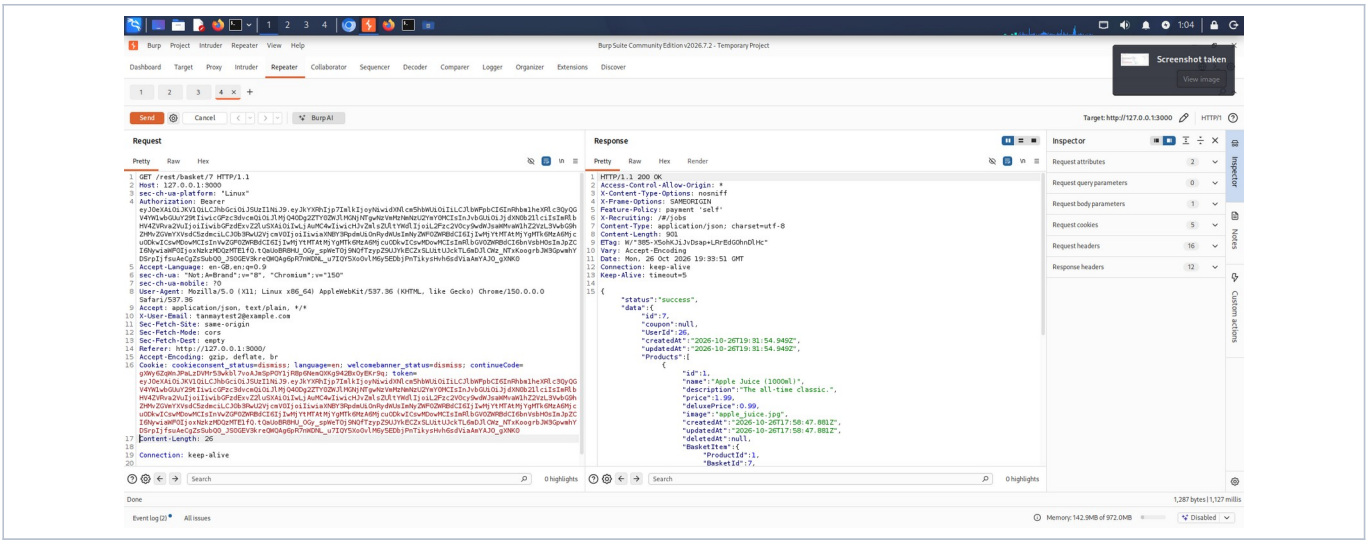


Figure 7 — Basket 7 returning UserId 26's cart contents

Assessment: Each basket ID returned data belonging to the account whose token was presented in that request — the evidence captured does ***not*** show one account's token successfully retrieving the *other* account's basket. Per the testing principle in Section 4, this was reported as **inconclusive, not confirmed vulnerable** at the time. GET /rest/basket/99999 (a non-existent basket) correctly returned data: null rather than an error leaking existence information.

Updated recommendation: The supplementary address-endpoint testing in F-8 below used the correct technique for this exact question — hold one account's token fixed and enumerate the ID path parameter — and confirmed an IDOR on a structurally similar endpoint. The basket endpoint should be retested with that same technique (fixed token, enumerate basket IDs sequentially) before this item can be closed as a confirmed negative.

F-7: Minor observations (informational)

- GET /api/Products/2?d=Tue%20Oct%2027%202026 — an arbitrary, undocumented query parameter (d) was accepted without validation or error; the product resource was returned normally. Low-risk, but indicates the endpoint does not reject unexpected parameters.

- GET /api/Addresss (intentionally probing a non-existent/misspelled route) returned 200 OK with an empty data: [] rather than a 404, which can help an attacker distinguish valid-but-empty collection endpoints from genuinely non-existent routes during reconnaissance.

SUPPLEMENTARY TESTING SESSION — ADDRESS ENDPOINT & ERROR HANDLING FOLLOW-UP

A follow-up Burp Suite session (27 October 2026, 13:06–20:13 local) specifically targeted the address-book feature with both test accounts to resolve the inconclusive basket IDOR question from F-6, and to probe additional error paths. Findings F-8 through F-10 below are the result.

F-8: Confirmed Insecure Direct Object Reference (IDOR) on the address endpoint (Broken Access Control)

Test setup: A new address was created for test account tanmaytest2026@example.com (UserId 25) via POST /api/Addresss/, which returned record id 8.

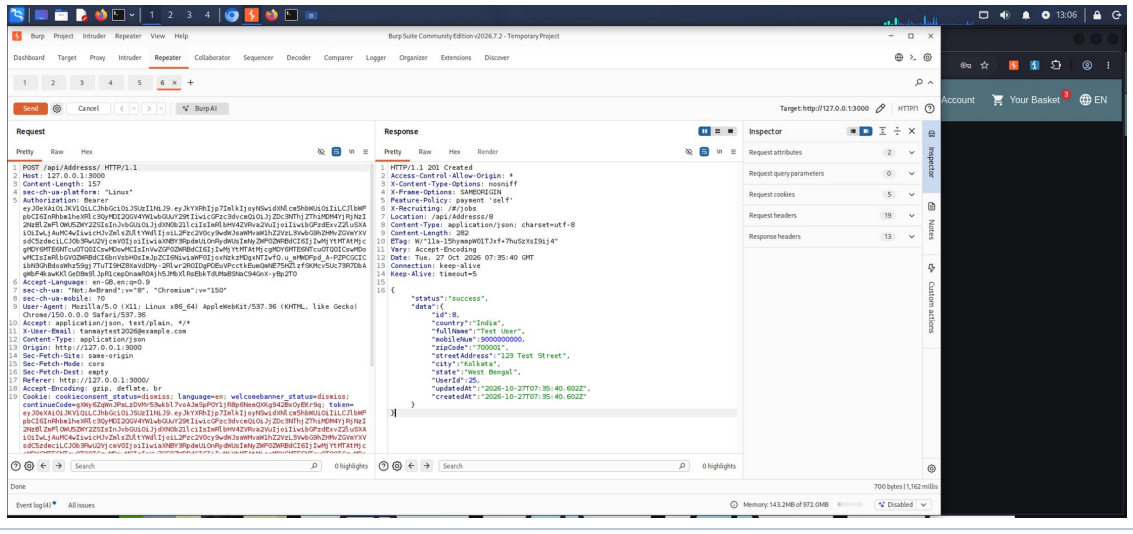


Figure 8 — POST /api/Addresss/ creating a new address record (id 8, UserId 25)

Request: Using the session for the second test account, tanmaytest2@example.com (UserId 26), two GET requests were sent back-to-back against /api/Addresss/{id} with no change to the Authorization Bearer token between them — only the numeric id path parameter was varied: first id 8, then id 7.

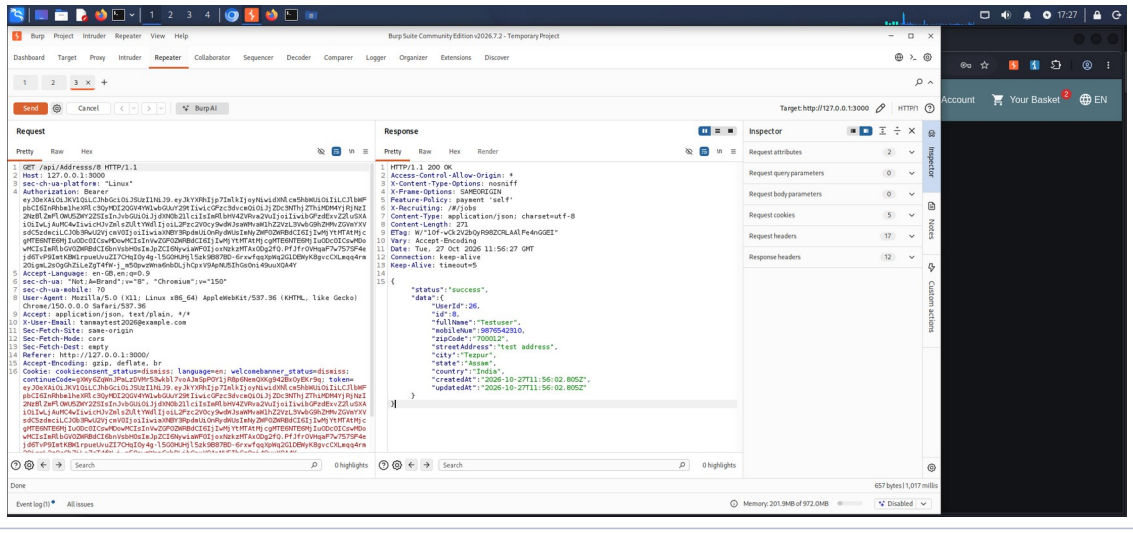


Figure 9 — GET /api/Addresss/8 under the UserId 26 session returns UserId 26's own record (baseline)

Response: The request for id 8 returned the requester's own record (UserId 26), as expected. The very next request, for id 7, returned **200 OK** with a complete address record — full name, mobile number, street address, city, state, and zip code — belonging to **UserId 25, a different account**, using the same session/token.

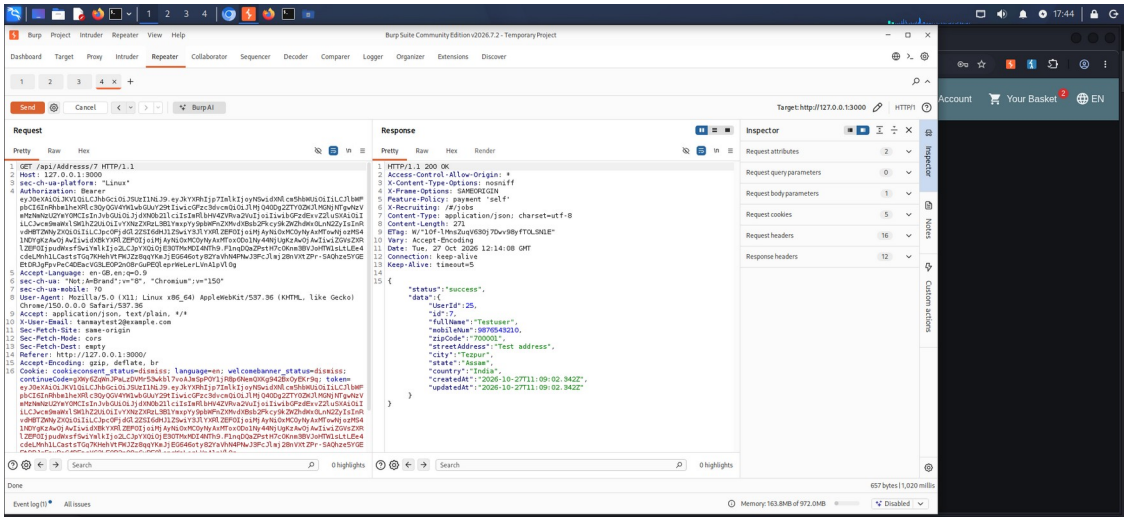


Figure 10 — GET /api/Addresss/7, same session, returns a different user's (UserId 25) full address record

Assessment: Confirmed **IDOR / Broken Access Control** (OWASP API Security Top 10 — API1:2023 Broken Object Level Authorization; CWE-639). The server does not verify that the address record's UserId matches the UserId carried in the caller's authenticated session before returning it. In practice, any authenticated user can enumerate sequential, low-cardinality address IDs and harvest other customers' personally identifiable information (name, phone number, full postal address) without any further privilege. This is the strongest finding of the engagement: unlike F-6, it is directly demonstrated with before/after evidence using a single fixed token against two different object IDs.

F-9: Inconsistent route-level protection between singular and misspelled address paths

Request: GET /api/Address/8 — the correctly spelled, singular form of the resource path — sent with the same Bearer token and headers used for the F-8 requests.

Response: 400 Bad Request — {"status":"error","data":"Malicious activity detected."}

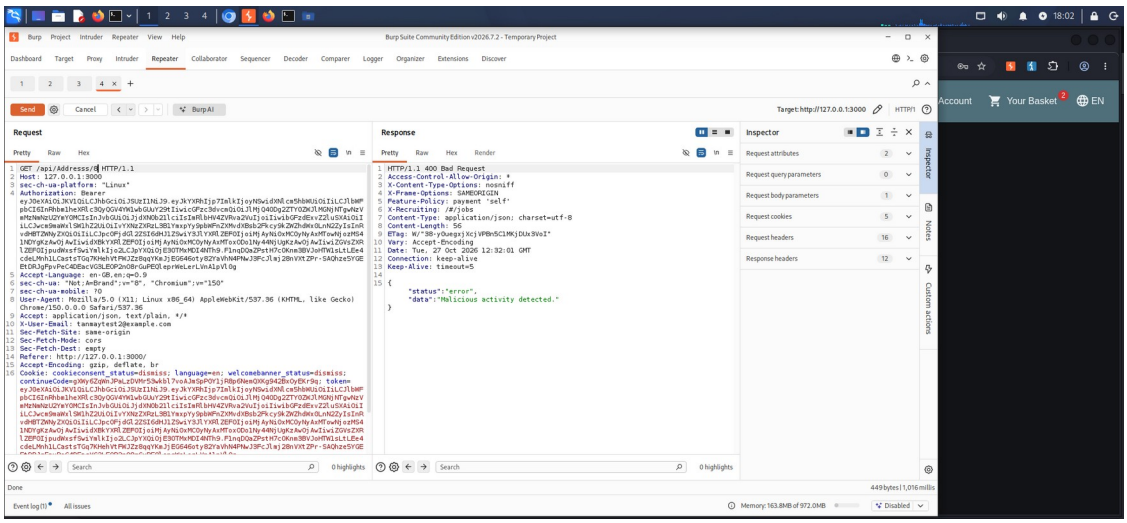


Figure 11 — GET /api/Address/8 (singular) rejected by an anomaly-detection check

Assessment: Informational / mixed-signal finding. The application does contain anomaly-detection logic that will outright reject some requests against an address-style path — but it is attached to a path variant (*/api/Address/**) that is not the one the real application actually calls. The path the client genuinely uses — */api/Addresss/** (note the extra "s") — carries no equivalent protection and is exactly the endpoint shown vulnerable to IDOR in F-8. This is a case of protective controls existing in the codebase but not being applied uniformly across all functionally equivalent routes, which gives a false sense of coverage. Recommend auditing the routing table for duplicate/aliased handlers and applying identical authorization and anomaly-detection middleware to every alias of a given resource.

F-10: Additional verbose stack-trace disclosure via a malformed basket path

Request: GET /rest/basket/ — a trailing-slash request with no basket ID supplied.

Response: 500 Internal Server Error with a full Node.js/Express stack trace: "Error: Unexpected path: /rest/basket/", including internal file paths (/juice-shop/build/routes/angular.js, /juice-shop/build/lib/utils.js, /juice-shop/build/lib/insecurity.js) and the complete Express router call chain (Layer.handle, trim_prefix, router.process_params).

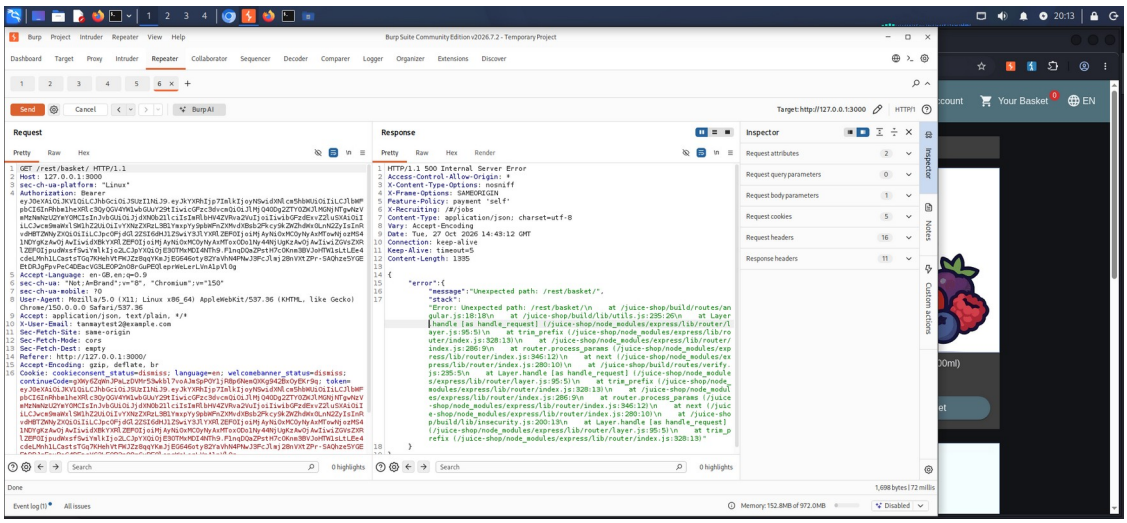


Figure 12 — 500 error on a malformed basket path, again exposing a full internal stack trace

Assessment: Extends F-2. A second, independent trigger — a malformed/incomplete path rather than a business-logic violation — again causes the server to return complete internal stack-trace detail to the client, confirming this is a **systemic error-handling gap** rather than a one-off. The disclosed paths confirm the backend is a Node.js/Express/Sequelize application rooted at /juice-shop/, which materially assists an attacker in fingerprinting the stack and locating further framework-specific vulnerabilities. Severity is assessed the same as F-2 (Medium).

6. Risk and Impact Summary

ID	Finding	Impact area	Severity
F-1	Admin configuration endpoint accessible with a standard user token	Confidentiality, Authorization	High
F-2	Verbose server error discloses Sequelize stack trace	Confidentiality (recon/fingerprinting)	Medium
F-3	Empty search query returns entire catalog; unescaped HTML in descriptions	Confidentiality, potential Integrity (Stored XSS risk)	Medium
F-4	Authorization header enforced on basket endpoint	— (positive control)	Informational
F-5	SQLi probe (') handled safely	— (positive control)	Informational
F-6	Basket object-level access (IDOR)	Confidentiality, Authorization	Inconclusive
F-7	Unvalidated extra query parameter; non-404 on misspelled route	Reconnaissance aid	Low
F-8	Confirmed IDOR on address endpoint — cross-account PII disclosure	Confidentiality, Authorization, PII exposure	High
F-9	Inconsistent anomaly-detection coverage across address route aliases	Authorization (false sense of coverage)	Informational
F-10	Second verbose stack-trace disclosure (malformed basket path)	Confidentiality (recon/fingerprinting)	Medium

Severity ratings follow the scale in the reference methodology: **Low** (minor impact), **Medium** (limited data exposure or weakened control), **High** (unauthorized access to sensitive data/functionality), **Critical** (full account takeover or system compromise). No Critical-severity finding was confirmed in this round of testing. F-1 and F-8 are each rated High: F-1 exposes internal application configuration and feature-flag data to any authenticated (non-privileged) user, and F-8 exposes another customer's name, phone number, and physical address through a directly demonstrated authorization bypass — both could materially assist, or directly constitute, further attacks against real users.

7. Remediation Recommendations

F-1 — Admin configuration endpoint

- Enforce server-side role/permission checks (e.g. isAdmin) on every /admin/* route, independent of whether a valid token is merely present.
- Return a generic 403 Forbidden for non-privileged callers, without revealing that the resource exists.
- Audit all other /admin/* and /rest/admin/* routes for the same gap.

F-2 & F-10 — Verbose error disclosure (two independent triggers)

- Catch SequelizeUniqueConstraintError and other ORM/database exceptions, as well as routing/path exceptions, explicitly and return a clean 400/404 with a generic, user-safe message.
- Disable stack-trace/debug output in any environment reachable by untrusted clients; log full details server-side only.
- Add a centralized error-handling middleware as the last layer in the Express chain so no unhandled exception — regardless of its cause — can reach the client in raw form.

F-3 — Unbounded search / HTML in data

- Reject or default empty search queries to a bounded/paginated result rather than the full catalog.
- Sanitize or encode any HTML stored in product/description fields before it reaches the API response, or ensure the frontend strictly escapes it on render; treat this as a Stored XSS candidate until client-side rendering is verified.

F-6 & F-8 — Object-level authorization (basket and address endpoints)

- Treat F-8 as proof-of-concept for a class of bug and immediately retest F-6 (basket) and any other /{resource}/{id} endpoint using the same fixed-token, varied-ID technique.
- On every request to an object by ID — baskets, addresses, orders, and any future resource — explicitly verify server-side that record.UserId matches the authenticated user extracted from the session/token; never trust the path parameter alone.
- Prefer resolving "the current user's own records" from the authenticated session rather than accepting an arbitrary object ID from the client where the use case allows it.
- Add automated authorization regression tests (e.g. an integration-test suite that, for every object-returning endpoint, asserts that User A's token cannot retrieve User B's record) so this class of bug is caught before release.

F-9 — Inconsistent route protection

- Identify and remove duplicate/aliased route handlers for the same resource (e.g. /api/Address vs /api/Addresss); route all traffic through a single, consistently protected handler.
- Apply identical authentication, authorization, and anomaly-detection middleware to every path that can reach a given resource, rather than relying on a single "canonical" path being the only one in use.

General / defense-in-depth (applies across findings)

- Treat all client-supplied data, headers, and tokens as untrusted; validate type, range, and authorization on every request.
- Apply centralized, automated access-control tests so that any new endpoint is checked for role enforcement and object-ownership enforcement before release.
- Apply consistent, generic error responses across the API, with verbose diagnostics routed to server-side logs only.

8. Conclusion

This engagement, conducted over an initial and a supplementary Burp Suite–based testing session against OWASP Juice Shop, confirmed **two High-severity Broken Access Control findings** — disclosure of the full admin application configuration to a non-privileged token (F-1), and a directly demonstrated cross-account IDOR on the address endpoint exposing another user's name, phone number, and postal address (F-8) — along with **two Medium-severity information-disclosure findings arising from the same systemic root cause** (verbose stack-trace leakage on two independent error triggers, F-2 and F-10), one further Medium-severity finding (unbounded catalog return on empty search, with an associated Stored XSS lead to verify, F-3), and informational observations around inconsistent route protection (F-9) and minor reconnaissance aids (F-7).

Two controls — authentication enforcement on the basket endpoint and resistance to a basic SQL-injection probe — were confirmed to be working correctly (F-4, F-5). The original basket IDOR test (F-6) remains formally inconclusive, but the confirmed IDOR on the structurally similar address endpoint (F-8) significantly raises the likelihood that the same gap exists in the basket resource and should be treated as a **priority retest**, not a closed item.

Remediation should be prioritized in this order: (1) **F-8** — add object-ownership checks to the address endpoint and retest F-6 using the same technique, given the direct, demonstrated exposure of customer PII; (2) **F-1** — enforce role checks on admin-scoped routes; (3) **F-2 / F-10** — centralize error handling to stop internal stack traces reaching clients; (4) remaining Medium/Low items. These are consistent with the defense-in-depth principles outlined in Section 7 and, for F-1, F-2, and F-10, are low-effort fixes with outsized risk reduction.